# CSCI 232 Assignment 07 – Shapes: Polymorphism and Your Own Insertion Sort

Points: 10

---

## Instructions

1. **Fork** the repository on GitHub: `https://github.com/jpach-cs/232-ass07-template`
2. **Clone** your forked repository to your local machine.
3. Open `src/code.cpp`, `src/tests.cpp` and `include/code.hpp`.
4. Uncomment the author block in `src/code.cpp` and fill in your name:

```
// std::string AUTHOR_NAME       = "Your Name";
// std::string AUTHOR_AUTHORSHIP = "I acknowledge that I have worked on this
// assignment independently...";
```

5. Work through each class in **three parts**:

### Part 1 — Class Hierarchy (`src/code.cpp`)

- Implement `Shape::operator<` (compare shapes by `area()`).
- Implement the constructors (member initializer list), `area()` and `name()` of `Circle`, `Square` and `Rectangle`.
- `Shape` is **abstract**: it is given in the header, do not change it.

### Part 2 — Class `ShapeList` (`src/code.cpp`)

- A wrapper around `std::list<Shape*>` that **owns** the shapes (RAII).
- Implement insertion, removal, queries and printing.
- Implement `insertionSort()` with **your own** stable insertion sort. The list must compare **shapes**, never pointers.
- The implementation stays hidden: no public method returns a `Shape*` or an iterator.

### Part 3 — Unit Tests (`src/tests.cpp`)

- Implement the 8 unit test bodies using `TEST_ASSERT_`

1. Build and run locally using C++20 (`g++`):

```
make clean
make
./run_tests
./check_rules.sh
```

7. When all local tests pass — **commit** and **push** to your repository.
8. Go to the **Actions** tab on GitHub and verify the CI results.

## Classes `Shape`, `Circle`, `Square`, `Rectangle` — `src/code.cpp` (Part 1)

`Shape` is an abstract class. `area()` and `name()` are pure virtual, the destructor is virtual (all given in `include/code.hpp`).

| Method | Description |
|---|---|
| `Shape::operator<` | Return `true` if this shape has a **strictly smaller** `area()` than `other`. Use the virtual `area()` of both sides |
| `Circle(double)` | Constructor — initialize `radius` with an initializer list |
| `Circle::area` | `pi * radius * radius` (use `std::numbers::pi` from `<numbers>`) |
| `Square(double)` | Constructor — initialize `side` with an initializer list |
| `Square::area` | `side * side` |
| `Rectangle(double, double)` | Constructor — initialize `width` and `height` with an initializer list |
| `Rectangle::area` | `width * height` |
| `name` (each class) | Return exactly `"Circle"`, `"Square"` or `"Rectangle"` |

---

## `ShapeList` Class Methods — `src/code.cpp` (Part 2)

The caller creates a shape with `new` and **hands it over** with `addShape`. From then on only `ShapeList` may `delete` it.

| Method | Description |
|---|---|
| `ShapeList()` | Given: `= default`, the list starts empty |
| `~ShapeList()` | Destructor — delete **every** shape (RAII), then clear the list |
| `addShape` | Append at the **end**. Return `-1` and change nothing if the pointer is `nullptr`, `0` on success |
| `removeFirst` | Delete the first shape and remove it from the list. Return `false` if the list is empty, `true` otherwise |
| `length` | Return the number of shapes |
| `totalArea` | Return the sum of `area()` of all shapes (`0.0` if empty) |

| Method | Description |
|---|---|
| `printList` | Print each shape on its own line: `name()`, one space, `area()`. Return the number of printed shapes (`0` for an empty list) |
| `areaAt` | `area()` of the shape at the 0-based index, or `std::nullopt` if the index is outside the list |
| `nameAt` | `name()` of the shape at the index, or `std::nullopt` |
| `insertionSort` | Sort **in place**, smallest area first, with **your own** insertion sort. Must be **stable** |
| `isSorted` | Return `true` if no shape is smaller than the one before it (empty and one-element lists are sorted) |

**Notes**

- `std::list` has **no** `operator[]`. To reach the element number `i`, use an iterator: `*std::next(data.begin(), i)`.
- The list holds `Shape*`, not `Shape`: `Shape` is abstract, and a derived object stored by value would be sliced.
- Be careful: for two `Shape*` pointers `a` and `b`, `a < b` compares **addresses**, exactly like pointers in C. Our overloaded `operator<` takes two **shapes**, so you must dereference: `*a < *b`.
- Inside the list, iterators are one level deeper: an iterator `it` points to a `Shape*`, so the shape itself is `**it`.
- **Stable** means: shapes with equal area keep their original order (`Square(2)` and `Rectangle(1, 4)` both have area 4). Move an element only while the previous one is **strictly** bigger than it.
- Idea for `insertionSort`: take the next element `currentIt`, remember `std::next(currentIt)` (the next round starts there), walk a second iterator left while the previous shape is bigger, then move the element with `data.splice(gapIt, data, currentIt)` (it moves a node in $\Theta(1)$ and copies nothing).
- Never `delete` a shape you gave to `addShape`: the list owns it, a second `delete` is a crash.

---

## Tests to implement — `src/tests.cpp` (Part 3)

| Test function | What to verify |
|---|---|
| `test_shape_areas` | `Circle(1)`, `Square(2)`, `Rectangle(2, 3)` have areas pi, 4 and 6 |

| Test function | What to verify |
|---|---|
| `test_shape_names_through_base_pointer` | An array of three `Shape*` returns the right `name()` through the **base** pointer; delete the shapes at the end |
| `test_shape_operator_less` | With `const Shape&` to `Square(1)` and `Square(2)`: `small < big` is true, `big < small` and `small < small` are false |
| `test_shape_equal_area_not_less` | `Square(2)` and `Rectangle(1, 4)`: neither `a < b` nor `b < a` |
| `test_list_add_and_length` | Two `addShape` calls return `0` and `length()` is `2` |
| `test_list_add_null_guard` | `addShape(nullptr)` returns `-1` and `length()` stays `0` |
| `test_insertionSort_orders_by_area` | `Rectangle(2, 5)`, `Circle(1)`, `Square(2)` become `Circle`, `Square`, `Rectangle`; `isSorted()` is true |
| `test_insertionSort_empty_and_single` | Sorting an empty list does not crash; a one-element list is unchanged |

Example:

```
void test_shape_equal_area_not_less(void)
{
    Square a(2);
    Rectangle b(1, 4);

    TEST_ASSERT_TRUE_MESSAGE(!(a < b), "equal areas: a < b must be false");
    TEST_ASSERT_TRUE_MESSAGE(!(b < a), "equal areas: b < a must be false");
}
```

Compare `double` values with a tolerance, never with `==`: `std::fabs(x - y) < 1e-9`.

---

## Rules

- Do **not** modify `include/code.hpp` or any function **signature**.
- All code must compile under **C++20** (`-std=c++20`).
- Write the sort yourself: `std::sort`, `std::stable_sort`, `std::list::sort` and `qsort` are **forbidden** (CI checks `src/code.cpp`, `check_rules.sh`).
- The sort must compare shapes with `Shape::operator<`. Do not hard-code the area formula in `ShapeList`.
- No public method may return a `Shape*` or an iterator.
- Always guard against `nullptr` before dereferencing a pointer.
- Every shape is deleted **exactly once**, inside `ShapeList`.

- CI builds with AddressSanitizer: memory leaks, double `delete` and use-after-free fail the build.
- CI runs your `tests.cpp` together with the instructor tests — both must pass.

---

## Grading

| Component | Points |
|---|---|
| Shape hierarchy (Part 1) | 3 |
| `ShapeList` class implementation (RAII, methods, own insertion sort) | 5 |
| All test cases in `tests.cpp` passing | 2 |
| **Total** | **10** |

---

## Author block — required

```
std::string AUTHOR_NAME       = "Your Name";
std::string AUTHOR_AUTHORSHIP = "I acknowledge that I have worked on this "
"assignment independently, except where explicitly noted and referenced. "
"Any collaboration or use of external resources has been properly cited. "
"I am fully aware of the consequences of academic dishonesty and agree "
"to abide by the university's academic integrity policy.";
```